



ENTENDER EL CÓDIGO

Los 5 archivos de la app ToDo, explicados como un cuento — ahora con el código real comentado

0 · El mapa (de dónde sale cada cosa)

index.html

lo que VES
la pantalla

js/app.js

lo que PIENSA el
front
pide y pinta

server.js

puerta del backend
conecta piezas

routes/tareas.js

el menú CRUD
la lógica

db.js

la memoria
la base de datos

Regla: el front solo pide y muestra. El back solo procesa. La BD solo guarda. Cada uno hace UNA cosa.

1 · index.html — la pantalla (solo estructura, sin lógica)

- **id="input-nombre"** → la etiqueta única (el "DNI") para que JS lo encuentre
- **required** → el navegador no deja enviar si está vacío
- **placeholder** → el texto gris de ayuda
- **type="submit"** → dispara el evento **submit** (JS lo atrapa con **preventDefault** y no recarga)
- **type="button"** → NO envía nada: solo ejecuta lo que JS le asigne (cancelar edición)

```
<!-- id = el "DNI" que usa JS para encontrar el campo -->
<input type="text" id="input-nombre" placeholder="Qué hay que hacer" required>

<!-- submit → JS lo atrapa y evita recargar la página -->
<button type="submit" id="btn-agregar">Agregar tarea</button>
<button type="button" id="btn-cancelar" class="hidden">Cancelar edición</button>

<!-- lista-tareas = acá app.js crea una tarjeta por cada tarea (HTML dinámico) -->
<div id="lista-tareas"></div>

<!-- JS SIEMPRE al final del body: para que el HTML ya exista cuando corre -->
<script src="js/app.js"></script>
```

El HTML es **estático**: existe al arrancar. Lo **dinámico** (las tarjetas de tareas) lo crea JS con **innerHTML** en el momento.

2 · js/app.js — el front (lo que PIENSA la pantalla)

El corazón: cargarTareas → pintarTareas

```
// fetch = "pedido al backend" · await = "esperá la respuesta"
async function cargarTareas() {
  const response = await fetch(API_URL); // GET → "traeme las tareas"
  const tareas = await response.json(); // JSON que llegó → objetos JS
  pintarTareas(tareas); // las dibuja en pantalla
}

// for...of = "por cada tarea de la lista"
for (const tarea of tareas) { // recorre el arreglo
  const div = document.createElement('div'); // crea una cajita
  div.innerHTML = `... ${tarea.nombre} ...`; // HTML dinámico
  // ${tarea.nombre} = el valor de ESE objeto de esta vuelta (no el último)
  listaTareas.appendChild(div); // la agrega a la lista
}
```

- **tarea.nombre** → la tarea es un OBJETO; accedés a sus campos con punto
- **contadores** → **tareas.length** (total) y **filter** (hechas) — nunca números fijos
- **data-id** en los botones → guarda el id de la tarea "para saber a cuál apuntar después"

Las 3 funciones que hablan con el backend

Función	Método	Para qué
agregarTarea(datos)	POST	manda la tarea nueva
modificarTarea(id, datos)	PUT	edita o cambia estatus
eliminarTarea(id)	DELETE	borra una tarea

Los "oídos" del front (eventos)

```
form.addEventListener('submit', ...) // apretás "Agregar" → preventDefault() evita recargar
listaTareas.addEventListener('click', ...) // tocás editar/eliminar (delegación de eventos)
listaTareas.addEventListener('change', ...) // cambiás el chip de estatus
```

Fuente única de verdad: después de cada operación se llama **cargarTareas()** → la lista SIEMPRE se reconstruye desde la BD, nunca se duplica el estado (Leo, clase 8).

3 · server.js — la puerta de entrada del backend

```
// Este archivo SOLO conecta piezas. No tiene lógica de negocio.
const express = require('express');           // las 3 famosas: Express, Fastify, NestJS
const cors = require('cors');                 // deja que el front (otro proyecto) le hable
const tareasRouter = require('./routes/tareas'); // el "menú" CRUD, en otro archivo

const app = express();                        // crea la aplicación
const PORT = 3000;                           // puerto donde escucha

// Middlewares = funciones que van "en el medio" de cada pedido
app.use(cors());                             // permite llamadas desde el frontend
app.use(express.json());                     // entiende los JSON del body

// "Todo lo que llegue a /api/tareas, que lo maneje tareasRouter"
app.use('/api/tareas', tareasRouter);

app.listen(PORT, ...);                       // queda escuchando pedidos
```

Es chiquito a propósito: **solo conecta las piezas**. Leo criticó el server de 380 líneas — el nuestro tiene 18.

4 · routes/tareas.js — el menú CRUD (el corazón)

```
// Los 3 personajes: req = lo que MANDAN · res = lo que RESPONDÉS · db = la memoria
// El patrón de CREAR (POST): 6 pasos
router.post('/', (req, res) => {
  const { nombre, responsable, ... } = req.body; // 1. saca lo que mandó el front
  if (!nombre || !responsable)
    return res.status(400).json({ error }); // 2. valida obligatorios (defensivo)
  if (!ESTADOS_VALIDOS.includes(estatus))
    return res.status(400).json({ error }); // 3. valida estatus cerrado
  db.prepare('INSERT INTO tareas ...').run(...); // 4. guarda en la BD
  const nuevaTarea = ... get(result.lastInsertRowid); // 5. relee con su id nuevo
  res.status(201).json(nuevaTarea); // 6. responde "creada"
});

// PUT = "si no mandan un campo, se CONSERVA el actual"
const nombreFinal = nombre ?? tarea.nombre; // "??" = si es null/undefined, usá lo que
había

// DELETE: "changes" dice cuántas filas afectó; 0 → no existía → 404
if (result.changes === 0) return res.status(404).json({ error });
res.status(204).end(); // 204 = "borrado, no tengo nada más que decirte"
```

Los códigos de respuesta (para sonar técnico)

Código	Qué significa	Cuándo
200	OK	leer / modificar con éxito
201	Created	crear con éxito
204	No Content	borrar con éxito (sin cuerpo)
400	Bad Request	datos inválidos (estatus chau, campos vacíos)
404	Not Found	la tarea no existe

ESTADOS_VALIDOS = la lista cerrada [pendiente, en_progreso, hecha]. Si llega otra cosa → 400. Eso es normalización (Leo, clase 6).

5 · db.js — la memoria (capa 3)

```
// SQLite viene INTEGRADO en Node 24 (sin instalar nada)
const db = new DatabaseSync('tareas.db'); // abre (o crea) el archivo
// IF NOT EXISTS = "solo la creo si no existe" (no pisa datos al re-arrancar)
db.exec(`CREATE TABLE IF NOT EXISTS tareas (
  id INTEGER PRIMARY KEY AUTOINCREMENT, // id lo genera SOLO la BD (Leo, clase 6)
  nombre TEXT NOT NULL, // qué hay que hacer (obligatorio)
  responsable TEXT NOT NULL, // quién la toma (obligatorio)
  estatus TEXT NOT NULL DEFAULT 'pendiente', // cerrado: pendiente | en_progreso | hecha
  descripcion TEXT DEFAULT '' // detalle / notas (opcional)
)`);
module.exports = db; // "acá está la conexión, úsela" → la recibe routes/tareas.js
```

- Nadie toca el archivo directamente: todo pasa por **db**

6 · El flujo completo (ejemplo real: "Agrego una tarea")

1. index.html: el form dispara **submit** → app.js lo atrapa (preventDefault, no recarga)
2. app.js: arma el objeto { nombre, responsable, estatus, descripcion } y hace **fetch POST** a `http://localhost:3000/api/tareas`
3. server.js: recibe → lo pasa a **routes/tareas.js**
4. routes/tareas.js: valida (400 si algo mal) → **db.prepare('INSERT...')** guarda en tareas.db
5. Responde JSON **201** → app.js llama **cargarTareas()** → **pintarTareas()** → ves la tarea nueva en pantalla

Mismo viaje para editar (PUT), cambiar estatus (PUT) y borrar (DELETE) — cambia solo la operación de la ruta.

7 · Tu ejercicio para el martes

Explicale a Leo (o a vos mismo) qué hace **routes/tareas.js** usando los 6 pasos del patrón POST. Si podés decirlo con tus palabras sin mirar, lo dominás.

- **¿Qué son req y res?** → lo que mandan / lo que respondés
- **¿Por qué el server es chico?** → solo conecta piezas (crítica de Leo al de 380 líneas)
- **¿Para qué sirve ESTADOS_VALIDOS?** → valores cerrados, normalización (clase 6)
- **¿Dónde está la fuente única de verdad?** → en la BD; el front siempre reconstruye desde el GET
- **¿Qué hace ?? (doble signo)?** → "si vino null/undefined, usá lo que ya había" (PUT defensivo)
- **¿Qué significa 204?** → "borrado, no tengo nada más que decirte"